

31 Transformer升级之路：8、长度外推性与位置鲁棒性

Jan By 苏剑林 | 2023-01-31 | 54857位读者

上一篇文章《Transformer升级之路：7、长度外推性与局部注意力》我们讨论了Transformer的长度外推性，得出的结论是长度外推性是一个训练和预测的不一致问题，而解决这个不一致的主要思路是将注意力局部化，很多外推性好的改进某种意义上都是局部注意力的变体。诚然，目前语言模型的诸多指标看来局部注意力的思路确实能解决长度外推问题，但这种“强行截断”的做法也许不符合某些读者的审美，因为人工雕琢痕迹太强，缺乏了自然感，同时也让人质疑它们在非语言模型任务上的有效性。

本文我们从模型对位置编码的鲁棒性角度来重新审视长度外推性问题，此思路可以在基本不对注意力进行修改的前提下改进Transformer的长度外推效果，并且还适用多种位置编码，总体来说方法更为优雅自然，而且还适用于非语言模型任务。

问题分析

在之前的文章中，我们分析了长度外推性的缘由，给出了“长度外推性是一个训练和预测的长度不一致的问题”的定位，具体不一致的地方则有两点：

- 1、预测的时候用到了没训练过的位置编码（不管绝对还是相对）；
- 2、预测的时候注意力机制所处理的token数量远超训练时的数量。

其中，第2点说的是更多的token会导致注意力更加分散（或者说注意力的熵变大），从而导致的训练和预测不一致问题，其实我们在《从熵不变性看Attention的Scale操作》已经初步讨论并解决了它，答案是将Attention从

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V \quad (1)$$

修改为

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{\log_m n}{\sqrt{d}} QK^\top \right) V \quad (2)$$

其中 m 是训练长度， n 是预测长度。经过这样修改（下面简称为“ $\log n$ 缩放注意力”），注意力的熵随着长度的变化更加平稳，缓解了这个不一致问题。个人的实验结果显示，至少在MLM任务上，“ $\log n$ 缩放注意力”的长度外推表现更好。

所以，我们可以认为第2点不一致性已经得到初步解决，那么接下来应该是先集中精力解决第1点不一致性。

随机位置

第1点不一致性，即“预测的时候用到了没训练过的位置编码”，那么为了解决它，就应该做到“训练阶段把预测所用的位置编码也训练一下”。一篇ACL22还在匿名评审的论文《Randomized Positional Encodings Boost Length Generalization of Transformers》首次从这个角度考虑了该问题，并且提出了解决方案。

论文的思路很简单：

随机位置训练 设 N 为训练长度（论文 $N = 40$ ）， M 为预测长度（论文 $M = 500$ ），那么选定一个较大 $L > M$ （这是一个超参，论文 $L = 2048$ ），训练阶段原本长度为 N 的序列对应的位置序列是 $[0, 1, \dots, N-2, N-1]$ ，现在改为从 $\{0, 1, \dots, L-2, L-1\}$ 中随机不重复地选 N 个并从小到大排列，作为当前序列的位置序列。

基于numpy的参考代码为：

```
1 def random_position_ids(N, L=2048):
2     """从[0, L)中随机不重复挑N个整数，并从小到大排列
3     """
4     return np.sort(np.random.permutation(L)[:N])
```

预测阶段，也可以同样的方式随机采样位置序列，也可以直接在区间中均匀取点（个人的实验效果显示均匀取点的效果一般好些），这就解决了预测阶段的位置编码没有被训练过的问题。不难理解，这是一个很朴素的训练技巧（下面称之为“随机位置训练”），目标是希望Transformer能对位置的选择更加鲁棒一些，但后面我们将看到，它能取得长度外推效果的明显提升。笔者也在MLM任务上做了实验，结果显示在MLM上也是有效果的，并且配合“ $\log n$ 缩放注意力”提升幅度更明显（原论文没有“ $\log n$ 缩放注意力”这一步）。

新的基准

很多相关工作，包括上一篇文章提到的各种Local Attention及其变体的方案，都以语言模型任务构建评测指标，但不管是单向GPT还是双向的MLM，它们都高度依赖局部信息（局域性），所以之前的方案很可能只是因为语言模型的局域性才有良好的外推表现，假如换一个非局域性的任务，效果可能就变差了。也许正因为如此，这篇论文的评测并非是常规的语言模型任务，而是Google去年在论文《Neural Networks and the Chomsky Hierarchy》专门提出的一个长度外泛化基准（下面简称该测试基准为“CHE基准”，即“Chomsky Hierarchy Evaluation Benchmark”），这给我们提供了理解长度外推的一个新视角。

这个基准包含多个任务，分为R（Regular）、DCF（Deterministic Context-Free）、CS（Context-Sensitive）三个级别，每个级别的难度依次递增，每个任务的简介如下：

Even Pairs，难度R，给定二元序列，如“aabba”，判断2-gram中ab和ba的总数是否为偶数，该例子中2-gram有aa、ab、bb、ba，其中ab和ba共有2个，即输出“是”，该题也等价于判断二元序列的首尾字符是否相同。

Modular Arithmetic (Simple)，难度R，计算由 $\{0, 1, 2, 3, 4\}$ 五个数和 $\{+, -, \times\}$ 三个运算符组成的算式的数值，并输出模5后的结果，比如输入 $1 + 2 - 4$ ，那么等于 -1 ，模5后等于4，所以输出4。

Parity Check，难度R，给定二元序列，如“aaabba”，判断b的数目是否为偶数，该例子中b的数目为2，那么输出“是”。

Cycle Navigation，难度R，给定三元序列，其中每个元分别代表 $+0$ 、 $+1$ 、 -1 之一，输出从0出发该序列最终的运算结果模5的值，比如0, 1, 2分别代表 $+0$ 、 $+1$ 、 -1 ，那么010211代表 $0 + 0 + 1 + 0 - 1 + 1 + 1 = 2$ ，模5后输出2。

Modular Arithmetic，难度DCF，计算由 $\{0, 1, 2, 3, 4\}$ 五个数、括号 $(,)$ 和 $\{+, -, \times\}$ 三个运算符组成的算式的数值，并输出模5后的结果，比如输入 $-(1 - 2) \times (4 - 3 \times (-2))$ ，那么结果为10，模5后等于0，所以输出0，相

比Simple版本，该任务多了“括号”，使得运算上更为复杂。

Reverse String，难度DCF，给定二元序列，如“aabba”，输出其反转序列，该例子中应该输出“abbaa”。

Solve Equation，难度DCF，给定由 $\{0, 1, 2, 3, 4\}$ 五个数、括号 $(,)$ 、 $\{+, -, \times\}$ 三个运算符和未知数 z 组成的方程，求解未知数 z 的数值，使得它在模5之下成立。比如 $-(1-2) \times (4-z \times (-2)) = 0$ ，那么 $z = 3$ ，解方程虽然看上去更难，但由于方程的构造是在Modular Arithmetic的基础上将等式中的某个数替换为 z ，所以保证有解并且解在 $\{0, 1, 2, 3, 4\}$ ，因此理论上我们可以通过枚举结合Modular Arithmetic来求解，因此它的难度跟Modular Arithmetic相当。

Stack Manipulation，难度DCF，给定二元序列，如“abbaa”，以及由“POP/PUSH a/PUSH b”三个动作组成的堆栈操作序列，如“POP / PUSH a / POP”，输出最后的堆栈结果，该例子中应该输出“abba”。

Binary Addition，难度CS，给定两个二进制数，输出它们的和的二进制表示，如输入10010和101，输出10111，注意，这需要都在字符层面而不是数值层面输入到模型中进行训练和预测，并且两个数字是串行而不是并行对齐地提供的（可以理解为输入的是字符串10010 + 101）。

Binary Multiplication，难度CS，给定两个二进制数，输出它们的和的二进制表示，如输入100和10110，输出1011000，同Binary Addition一样，这需要都在字符层面而不是数值层面输入到模型中进行训练和预测，并且两个数字是串行而不是并行对齐地提供的（可以理解为输入的是字符串100 × 10110）。

Compute Sqrt，难度CS，给定一个二进制数，输出它的平方根的下取整的二进制表示，如输入101001，那么输出结果为 $\lfloor \sqrt{101001} \rfloor = 101$ ，这个难度同Binary Multiplication，因为至少我们可以从0到所给数结合Binary Multiplication逐一枚举来确定结果。

Duplicate String，难度CS，给定一个二元序列，如“abaab”，输出重复一次后的序列，该例子应该输出“abaabaabaab”，这个简单的任务看上去是难度R，但实际上是CS，大家可以想想为什么。

Missing Duplicate，难度CS，给定一个带有缺失值的二元序列，如“ab_aba”，并且已知原始的完整序列是一个重复序列（上一个任务的Duplicate String），预测确实值，该例子应该输出a。

Odds First，难度CS，给定一个二元序列 $t_1 t_2 t_3 \cdots t_n$ ，输出 $t_1 t_3 t_5 \cdots t_2 t_4 t_6 \cdots$ ，如输入aaabaa，将输出aaaaba。

Bucket Sort，难度CS，给定一个 n 元数值序列（数列中的每个数都是给定的 n 个数之一），返回其从小到大排序后的序列，如输入421302214应该输出011222344。

可以看到，这些任务都具有一个共同特点，就是它们的运算都有固定的简单规则，并且理论上输入都是不限长度的，那么我们可以通过短序列来训练，然后测试在短序列上的训练结果能否推广到长序列中。也就是说，它可以作为长度外推性的一个很强的测试基准。

实验结果

首先，介绍原始论文《Neural Networks and the Chomsky Hierarchy》的实验结果，它对比了若干RNN模型及Transformer模型的效果（评测指标是诸位字符串的平均准确率，而不是整体结果的全对率）：

Level	Task	RNN	Stack-RNN	Tape-RNN	Transformer	LSTM
R	Even Pairs	100.0	100.0	100.0	96.4	100.0
	Modular Arithmetic (Simple)	100.0	100.0	100.0	24.2	100.0
	Parity Check [†]	100.0	100.0	100.0	52.0	100.0
	Cycle Navigation [†]	100.0	100.0	100.0	61.9	100.0
DCF	Stack Manipulation	56.0	100.0	100.0	57.5	59.1
	Reverse String	62.0	100.0	100.0	62.3	60.9
	Modular Arithmetic	41.3	96.1	95.4	32.5	59.2
	Solve Equation [°]	51.0	56.2	64.4	25.7	67.8
CS	Duplicate String	50.3	52.8	100.0	52.8	57.6
	Missing Duplicate	52.3	55.2	100.0	56.4	54.3
	Odds First	51.0	51.9	100.0	52.8	55.6
	Binary Addition	50.3	52.7	100.0	54.3	55.5
	Binary Multiplication [×]	50.0	52.7	58.5	52.2	53.1
	Compute Sqrt	54.3	56.5	57.8	52.4	57.5
	Bucket Sort ^{†★}	27.9	78.1	70.7	91.9	99.3

若干模型在若干长度外推测试任务上的效果对比

结果可能会让人意外，“风头正盛”的Transformer的长度外推效果是最差的（这里的Transformer还测试了不同的位置编码，并在每种任务上取了最优值），最好的是Tape-RNN。论文给它们的评级如下：

Transformer

<

RNN

<

LSTM

<

Stack-RNN

<

Tape-RNN

R⁻

R

R⁺

DCF

CS

而前面介绍的《Randomized Positional Encodings Boost Length Generalization of Transformers》所提的随机位置训练方法，则为Transformer挽回了一些劣势：

Level	Task	None	sin / cos	Relative	ALiBi	RoPE	Learned	Randomized (Ours)				
								sin / cos	Relative	ALiBi	RoPE	Learned [★]
R	EVEN PAIRS	50.4	50.9	96.4	67.3	51.0	50.7	100.0	100.0	81.5	100.0	97.5
	MODULAR ARITHMETIC (SIMPLE)	20.1	20.5	21.8	24.2	21.6	20.2	25.7	28.1	21.2	25.5	21.1
	PARITY CHECK [†]	51.9	50.5	51.8	51.7	51.3	50.3	52.6	52.2	50.3	52.3	52.6
	CYCLE NAVIGATION [†]	61.9	26.3	23.0	37.6	23.6	24.2	59.0	58.8	29.8	73.6	49.7
DCF	STACK MANIPULATION	50.3	50.1	53.6	57.5	51.2	49.2	72.8	77.9	70.6	68.2	69.1
	REVERSE STRING	52.8	50.6	58.3	62.3	51.9	50.7	75.6	95.1	77.1	69.9	52.9
	MODULAR ARITHMETIC	31.0	28.3	30.3	32.5	25.1	25.1	33.8	34.9	31.3	32.7	31.9
	SOLVE EQUATION	20.1	21.0	23.0	25.7	23.1	20.4	24.5	28.1	22.0	24.5	22.1
CS	DUPLICATE STRING	52.8	50.7	51.7	51.3	50.9	50.8	72.4	75.1	68.9	68.9	53.0
	MISSING DUPLICATE	52.5	51.3	54.0	54.3	56.5	51.0	52.5	100.0	79.7	88.7	52.7
	ODDS FIRST	52.8	51.6	52.7	51.4	51.3	50.6	65.9	69.3	64.7	65.6	52.7
	BINARY ADDITION	50.1	49.8	54.3	51.4	50.4	49.8	64.4	64.5	56.2	60.2	61.7
	BINARY MULTIPLICATION	49.9	50.1	52.2	51.0	50.2	49.6	52.1	50.1	50.5	51.7	51.9
	COMPUTE SQRT	50.2	50.1	52.4	50.9	50.5	50.2	52.5	53.3	51.2	52.3	52.0
	BUCKET SORT [†]	23.7	30.1	91.9	38.8	30.6	25.9	100.0	100.0	99.6	99.6	99.5

不同位置编码的Transformer在有随机位置训练下的长度外推效果对比

可以看到，在随机位置训练之下，不管哪种位置编码的Transformer都有明显的提升，这就进一步验证了上一篇文章的结论，即长度外推性跟位置编码本身的设计没太大关系。特别地，随机位置训练还在Bucket Sort这个任务上首次取得了全对的准确率。尽管在总体表现上依然欠佳，但这相比之前的结果已经算是长足进步了（不知道结合“log n 缩放注意力”能否有提升？）。值得注意的地方还有，上表显示ALIBI这个在语言模型任务中表现良好的方法，在CHE基准上并无表现出什么优势，尤其是加入随机位置训练后，它的平均指标比RoPE还差，这就初步肯定了前面的猜测，即各种Local Attention变体的方法表现良好，大概率是因为基于语言模型的评测任务本身有严重的局域性，对于非局域性的CHE基准，这些方法并没有优势。

原理反思 #

细思之下，“随机位置训练”会很让人困惑。简单起见，我们不妨设 $L = 2048$, $N = 64$, $M = 512$ ，这样一来，训练阶段所用的平均位置序列大致为 $[0, 32, 64, \dots, 2016]$ ，预测阶段所用的平均位置序列是 $[0, 4, 8, \dots, 2044]$ ，训练阶段和预测阶段的相邻位置差不一样，这也可以说是某种不一致性，但它表现依然良好，这是为什么呢？

我们可以从“序”的角度去理解它。由于训练阶段的位置id是随机采样的，那么相邻位置差也是随机的，所以不管是相对位置还是绝对位置，模型不大可能通过精确的位置id来获取位置信息，取而代之的是一个模糊的位置信号，更准确地说，是通过位置序列的“序”来编码位置而不是通过位置id本身来编码位置。比如，位置序列 $[1, 3, 5]$ 跟 $[2, 4, 8]$ 是等价的，因为它们都是从小到大排列的一个序列，随机位置训练“迫使”模型学会了一个等价类，即所有从小到大排列的位置序列都是等价的，都可以相互替换，这是位置鲁棒性的真正含义。

然而，笔者自己在MLM上做的实验结果显示，这个“等价类”的学习对模型还是有一定的困难的，更理想的方法是训练阶段依然使用随机位置，使得预测阶段的位置编码也被训练过，但是预测阶段的位置序列前面部分应该跟随机位置的平均结果一致。还是刚才的例子，如果预测阶段所用的位置序列是 $[0, 4, 8, \dots, 2044]$ ，那么我们希望训练阶段的随机位置平均结果是 $[0, 4, 8, \dots, 252]$ （即序列 $[0, 4, 8, \dots, 2044]$ 的前 N 个），而不是 $[0, 32, 64, \dots, 2016]$ ，这样训练和预测的一致性就更加紧凑了。

延伸推广

于是，笔者考虑了如下思路：

等均值随机位置训练 设 n 服从一个均值为 N 、采样空间为 $[0, \infty)$ 的分布，那么训练阶段随机采样一个 n ，然后从 $[0, n]$ 中均匀取 N 个点作为位置序列。

参考代码为：

```
1 def random_position_ids(N):
2     """先随机采样n，然后从[0, n]均匀取N个点
3     """
4     n = sample_from_xxx()
5     return np.linspace(0, 1, N) * n
```

注意，这样采样出来的位置序列是浮点数，因此不适用于离散的训练式位置编码，只适用于函数式位置编码如Sinusoidal或RoPE，下面假设只考虑函数式位置编码。

该思路的最大问题是如何选择适合的采样分布。笔者的第一反应是泊松分布，但考虑到泊松分布的均值和方差都是 n ，那么按照“ 3σ 法则”来估算，它只能外推到 $n + 3\sqrt{n}$ 长度，这显然太短了。经过挑选和测试，笔者发现有两个分布比较适合：一个是指数分布，它的均值和标准差都是 n ，那么即便按照“ 3σ 法则”，也能外推到 $4n$ 的长度，是一个比较理想的范围（实际还更长些）；另一个是beta分布，它定义在 $[0, 1]$ 上，我们可以将测试长度作为1，那么训练长度就是 $N/M \in (0, 1)$ ，beta分布有两个参数 α, β ，其中均值为 $\frac{\alpha}{\alpha+\beta}$ ，那么确保均值等于 N/M 后，我们还有额外的自由度控制1附近的概率，适合想要进一步拓展外推范围的场景。

笔者的实验结果显示，“等均值随机位置训练”结合“log n 缩放注意力”，在MLM任务上，能取得最佳的外推效果（训练长度64，测试长度512，采样分布为指数分布）。因为之前没做过CHE基准，所以笔者一时之间也没法测试CHE基准的效果，只能留到后面有机会再尝试了。

文章小结

本文从位置鲁棒性的角度思考了Transformer的长度外推性，得到了“随机位置训练”等增强长度外推性的新方案。同时，我们介绍了新的“CHE基准”，相比常规的语言模型任务，它具备更强的非局域性，可以更有效地评估长度外推相关工作。在它之下，之前的注意力局部化相关方法并没有较为突出的表现，相比之下“随机位置训练”效果更佳，这提醒我们应当在更全面的任务上评估相关方法的有效性，而不单单局限于语言模型任务。

转载到请包括本文地址：<https://spaces.ac.cn/archives/9444>
更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Jan. 31, 2023). 《Transformer升级之路：8、长度外推性与位置鲁棒性》 [Blog post]. Retrieved from <https://spaces.ac.cn/archives/9444>

```
@online{kexuefm-9444,
  title={Transformer升级之路：8、长度外推性与位置鲁棒性},
  author={苏剑林},
  year={2023},
  month={Jan},
  url={\url{https://spaces.ac.cn/archives/9444}},
}
```